



Oracle SQL – Fondamenti e Query Base

Corso introduttivo per sviluppatori e analisti · Giornata 1

🔑 Obiettivo

Scrivere le prime query SQL su Oracle Database

📖 Approccio

Teoria + esempi pratici + esercitazioni guidate

👥 Target

Sviluppatori e analisti a livello base

Introduzione alla Giornata

Benvenuti al corso **Oracle SQL – Fondamenti e Query Base**. Questa prima giornata è progettata per fornire le basi teoriche e pratiche indispensabili per lavorare con Oracle Database in modo efficace e autonomo.

Obiettivi di Apprendimento

-  Comprendere il modello relazionale e la struttura di un database
-  Conoscere l'ambiente Oracle Database e i suoi oggetti principali
-  Scrivere query SQL di base con SELECT, WHERE e ORDER BY
-  Applicare subito i concetti tramite esercitazioni pratiche guidate

Agenda della Giornata

- 01

Database relazionali

Concetti fondamentali: tabelle, righe, colonne, chiavi
- 02

Oracle Database

Panoramica dell'ambiente, schema e oggetti principali
- 03

Istruzione SELECT

Sintassi base, selezione colonne, alias
- 04

Filtri e ordinamento

Clausole WHERE e ORDER BY con esempi pratici
- 05

Esercitazioni

Scrittura autonoma di query su dataset di esempio

 Il corso alterna **moduli teorici** a **sessioni pratiche** al computer. Ogni concetto viene immediatamente consolidato con esempi reali su Oracle Database.

Cos'è un Database Relazionale?

Un **database relazionale** organizza i dati in **tabelle strutturate** collegate tra loro, superando i limiti dei tradizionali sistemi basati su file.

Database vs File

File tradizionale	Database relazionale
Dati duplicati e inconsistenti	Dati centralizzati e coerenti
Ricerche manuali e lente	Query veloci e strutturate
Nessun controllo accessi	Gestione granulare dei permessi
Difficile condivisione	Accesso multi-utente concorrente

Concetti Chiave

Tabella Struttura che raccoglie dati omogenei (es. tutti i dipendenti)	Riga (Record) Una singola occorrenza dell'entità (es. un dipendente specifico)
Colonna (Campo) Un attributo del dato (es. nome, cognome, stipendio)	

Esempio: Tabella DIPENDENTI

ID	NOME	REPARTO	STIPENDIO
101	Rossi Mario	IT	3.200
102	Bianchi Laura	HR	2.900
103	Verdi Luca	IT	3.500
104	Neri Anna	Finance	3.100

Le Chiavi

Primary Key (PK)

Identifica in modo univoco ogni riga della tabella. Non può essere NULL né duplicata. Nell'esempio: **ID** è la PK della tabella DIPENDENTI.

Foreign Key (FK)

Collega due tabelle tra loro creando una relazione. Es. il campo **REPARTO_ID** in DIPENDENTI può riferirsi alla tabella REPARTI.

Introduzione a Oracle Database

Cos'è Oracle Database?

Oracle Database è uno dei sistemi di gestione di basi di dati relazionali (RDBMS) più diffusi e affidabili al mondo. Sviluppato da Oracle Corporation, è utilizzato da grandi aziende, banche, enti pubblici e organizzazioni in tutto il mondo per gestire dati critici con alte prestazioni, sicurezza e disponibilità.



Storage affidabile

Archivia dati strutturati con garanzie ACID (Atomicità, Consistenza, Isolamento, Durabilità)



Sicurezza avanzata

Gestione granulare di utenti, ruoli e privilegi di accesso ai dati



Prestazioni elevate

Ottimizzatore di query e indicizzazione per interrogazioni veloci anche su grandi volumi di dati

Schema e Oggetti Principali

In Oracle, uno **schema** è un contenitore logico associato a un utente del database. Ogni schema può contenere diversi tipi di oggetti:

TABLE

Struttura base per i dati

VIEW

Query salvata come oggetto virtuale

INDEX

Accelera le ricerche sulle tabelle

SEQUENCE

Genera valori numerici univoci

Schema HR – Il Nostro Dataset di Esempio

Durante il corso utilizzeremo lo schema **HR (Human Resources)**, uno schema di esempio incluso in Oracle. Contiene tabelle realistiche come:

- `EMPLOYEES` – dati dei dipendenti
- `DEPARTMENTS` – reparti aziendali
- `JOB`s – mansioni e fasce retributive
- `LOCATIONS` – sedi geografiche



Per connettersi ad Oracle si usa tipicamente **SQL*Plus**, **SQL Developer** o **Oracle Live SQL**. Durante il corso utilizzeremo SQL tool di Apex per la sua interfaccia grafica intuitiva.

Sintassi Base: SELECT

L'istruzione `SELECT` è il comando fondamentale di SQL per **interrogare un database** e recuperare dati da una o più tabelle. Ogni query SQL inizia sempre con questa parola chiave.

Struttura della Query

La forma minima di una query `SELECT` è composta da due parti obbligatorie:

```
SELECT colonna1, colonna2, ...  
FROM   nome_tabella;
```

Il punto e virgola `;` termina l'istruzione. In Oracle è obbligatorio quando si esegue da SQL*Plus o script.

Selezionare Tutte le Colonne

Il carattere `*` (asterisco) è un jolly che restituisce **tutte le colonne** della tabella:

```
SELECT *  
FROM   employees;
```

⚠ Usare `SELECT *` è comodo in fase di esplorazione, ma in produzione è preferibile elencare le colonne esplicitamente per motivi di leggibilità e performance.

Selezione di Colonne Specifiche

Elencare solo le colonne necessarie rende la query più efficiente e l'output più leggibile:

```
SELECT first_name,  
       last_name,  
       salary  
FROM   employees;
```

Regole Sintattiche Fondamentali

Case Insensitive

SQL non è sensibile alle maiuscole per parole chiave e nomi oggetti: `SELECT` e `select` sono equivalenti

Spazi e Rientri

La formattazione del codice non influisce sull'esecuzione ma migliora enormemente la leggibilità

Nomi con Spazi

Se un nome di colonna contiene spazi o caratteri speciali, va racchiuso tra doppi apici: `"My Column"`

Commenti

Commento su riga: `-- testo`. Commento su più righe: `/* testo */`

Esempi Pratici di SELECT

Vediamo alcuni esempi concreti utilizzando la tabella `EMPLOYEES` dello schema HR, con output atteso e spiegazione riga per riga.

Esempio 1 – Query Base

```
SELECT employee_id,  
       first_name,  
       last_name,  
       department_id  
FROM   employees;
```

Output atteso: Restituisce una riga per ogni dipendente con le quattro colonne specificate. Il risultato contiene tutti i dipendenti presenti nella tabella (107 righe nello schema HR standard).

Esempio 2 – Alias di Colonna

Gli **alias** permettono di rinominare le colonne nell'output, rendendo i risultati più leggibili. Si usano con la parola chiave `AS` (opzionale):

```
SELECT first_name AS "Nome",  
       last_name  AS "Cognome",  
       salary     AS "Stipendio (€)"  
FROM   employees;
```

Output atteso: Le intestazioni delle colonne appariranno come *Nome*, *Cognome* e *Stipendio (€)* invece dei nomi originali del database. Gli apici doppi sono necessari quando l'alias contiene spazi o caratteri speciali.

Esempio 3 – Espressioni nel SELECT

È possibile includere espressioni aritmetiche direttamente nella clausola SELECT:

```
SELECT first_name,  
       salary,  
       salary * 12 AS stipendio_annuo,  
       salary * 1.1 AS "Stip. + 10%"  
FROM   employees;
```

Output atteso: Oltre al nome e allo stipendio mensile, vengono calcolate e mostrate due colonne derivate: lo stipendio annualizzato e quello con un incremento del 10%, senza modificare i dati nel database.

Esempio 4 – Concatenazione di Testo

```
SELECT first_name || ' ' || last_name  
       AS "Nome Completo",  
       email  
FROM   employees;
```

L'operatore `||` è l'operatore di concatenazione in Oracle SQL. Unisce due o più stringhe in un'unica colonna nell'output.

✔ Tutti questi esempi possono essere eseguiti immediatamente in SQL Developer collegandosi allo schema HR. Provate a modificare le colonne selezionate per esplorare la tabella!

Filtrare i Dati: Clausola WHERE

La clausola `WHERE` permette di **selezionare solo le righe** che soddisfano una condizione specifica, riducendo il numero di risultati restituiti dalla query.

Operatori di Confronto

Operatore	Significato	Esempio
<code>=</code>	Uguale a	<code>salary = 3000</code>
<code>!=</code> o <code><></code>	Diverso da	<code>dept_id <> 10</code>
<code>></code>	Maggiore di	<code>salary > 5000</code>
<code><</code>	Minore di	<code>salary < 3000</code>
<code>>=</code>	Maggiore o uguale	<code>salary >= 2500</code>
<code><=</code>	Minore o uguale	<code>hire_date <= '01/01/2005'</code>

Operatori Logici

AND

Entrambe le condizioni devono essere vere

OR

Almeno una condizione deve essere vera

NOT

Nega la condizione che segue

Operatori Speciali

BETWEEN ... AND ...

Verifica che un valore rientri in un intervallo (estremi inclusi).

```
salary BETWEEN 2000 AND 5000
```

IN (...)

Verifica che un valore sia presente in una lista di valori specificati.

```
department_id IN (10, 20, 90)
```

LIKE

Confronto su pattern di testo. Usa `%` (qualsiasi sequenza) e `_` (un singolo carattere).

```
last_name LIKE 'S%'
```

IS NULL / IS NOT NULL

Verifica la presenza o assenza di valori NULL (valore sconosciuto o mancante).

```
commission_pct IS NOT NULL
```



Attenzione: Non usare mai `= NULL` per confrontare i NULL. In SQL, NULL non è uguale a nulla, nemmeno a se stesso. Usare sempre `IS NULL` o `IS NOT NULL`.

Esempi Pratici con WHERE

Esempi progressivi di utilizzo della clausola WHERE, dai filtri più semplici alle condizioni combinate, tutti eseguibili sullo schema HR.

Filtri Semplici

```
-- Dipendenti del reparto 90
SELECT first_name, last_name,
       department_id
FROM   employees
WHERE  department_id = 90;

-- Stipendi superiori a 10.000
SELECT first_name, last_name, salary
FROM   employees
WHERE  salary > 10000;
```

Utilizzo di BETWEEN e IN

```
-- Stipendio tra 3000 e 6000 (inclusi)
SELECT first_name, last_name, salary
FROM   employees
WHERE  salary BETWEEN 3000 AND 6000;

-- Dipendenti in più reparti specifici
SELECT first_name, last_name,
       department_id
FROM   employees
WHERE  department_id IN (10, 20, 30, 90);
```

Filtri Combinati con AND / OR

```
-- Reparto 60 E stipendio > 4000
SELECT first_name, last_name,
       salary, department_id
FROM   employees
WHERE  department_id = 60
AND    salary > 4000;

-- Reparto 10 OPPURE reparto 20
SELECT first_name, last_name,
       department_id
FROM   employees
WHERE  department_id = 10
OR     department_id = 20;
```

Pattern Matching con LIKE

```
-- Cognome che inizia con 'S'
SELECT first_name, last_name
FROM   employees
WHERE  last_name LIKE 'S%';

-- Cognome che contiene 'an' in qualsiasi posizione
SELECT first_name, last_name
FROM   employees
WHERE  last_name LIKE '%an%';

-- Cognome con 'a' come seconda lettera
SELECT first_name, last_name
FROM   employees
WHERE  last_name LIKE '_a%';
```



L'operatore **AND** ha precedenza su **OR**. Usare le parentesi tonde () per rendere esplicita la logica e evitare comportamenti inattesi nelle condizioni complesse.



Ricorda: **%** sostituisce zero o più caratteri qualsiasi, **_** sostituisce esattamente un carattere. LIKE è case-sensitive in Oracle per impostazione predefinita.

Ordinamento dei Risultati: ORDER BY

La clausola `ORDER BY` permette di **ordinare le righe restituite** dalla query secondo uno o più criteri. Va sempre posizionata come **ultima clausola** dell'istruzione `SELECT`.

Sintassi e Direzione

```
SELECT colonna1, colonna2
FROM   tabella
WHERE  condizione
ORDER BY colonna1 [ASC | DESC];
```

ASC

Ordine crescente (default). Per numeri: dal più piccolo al più grande. Per testo: dalla A alla Z.

DESC

Ordine decrescente. Per numeri: dal più grande al più piccolo. Per testo: dalla Z alla A.



In Oracle, i valori **NULL** con `ORDER BY ASC` appaiono per ultimi; con `ORDER BY DESC` appaiono per primi. Si può modificare questo comportamento con `NULLS FIRST` o `NULLS LAST`.

Esempi di Ordinamento

```
-- Ordine alfabetico per cognome (ASC default)
SELECT first_name, last_name
FROM   employees
ORDER BY last_name;
```

```
-- Stipendio dal più alto al più basso
SELECT first_name, last_name, salary
FROM   employees
ORDER BY salary DESC;
```

```
-- Ordinamento su più colonne
-- Prima per reparto (crescente),
-- poi per stipendio (decrescente)
SELECT first_name, last_name,
       department_id, salary
FROM   employees
ORDER BY department_id ASC,
       salary DESC;
```

Ordinamento per Posizione o Alias

```
-- Usando la posizione della colonna nel SELECT
SELECT first_name, last_name, salary
FROM   employees
ORDER BY 3 DESC; -- ordina per salary (3ª colonna)
```

```
-- Usando l'alias definito nel SELECT
SELECT first_name, salary * 12 AS ann_salary
FROM   employees
ORDER BY ann_salary DESC;
```



Ordinare per posizione numerica è sconsigliato nel codice di produzione perché fragilizza la query: aggiungere una colonna cambia i numeri e modifica l'ordinamento in modo silenzioso.

Esercitazioni Pratiche

È ora di mettere in pratica tutto ciò che abbiamo imparato! Le esercitazioni sono strutturate in tre livelli di difficoltà crescente. Tutte le query utilizzano lo schema **HR** di Oracle.

Livello 1 – SELECT Base

1

Esercizio 1

Seleziona `first_name`, `last_name` e `email` da tutti i dipendenti.

2

Esercizio 2

Visualizza tutti i campi della tabella `departments`.

3

Esercizio 3

Mostra nome, cognome e stipendio annuo (`salary * 12`) con alias `stipendio_annuo`.

Livello 2 – Filtri WHERE

1

Esercizio 4

Trova tutti i dipendenti con `department_id = 50` e stipendio maggiore di 3.500.

2

Esercizio 5

Lista dei dipendenti assunti tra il 2000 e il 2005 (usa `hire_date` e `BETWEEN`).

3

Esercizio 6

Trova tutti i dipendenti il cui cognome contiene la lettera 'a' e appartengono ai reparti 20, 30 o 40.

Livello 3 – Ordinamento

1

Esercizio 7

Elenca i dipendenti del reparto 80 ordinati per stipendio decrescente. Mostra nome, cognome e stipendio.

2

Esercizio 8

Mostra tutti i dipendenti ordinati prima per `department_id` crescente, poi per `last_name` alfabetico.

3

Esercizio 9 ★

Trova i 5 dipendenti con lo stipendio più alto usando `WHERE` e `ORDER BY`. (Sfida: come limiteresti i risultati a 5 righe in Oracle?)

Dataset di Riferimento

EMPLOYEES

107 righe · 11 colonne

DEPARTMENTS

27 righe · 4 colonne

JOBS

19 righe · 4 colonne

 **Suggerimento:** Inizia sempre con `SELECT *` per esplorare la struttura di una tabella sconosciuta prima di scrivere query più specifiche.